Roteiro do vídeo — ThriveQueue (até 5 min)

Meta: 4min40s, para sobrar folga. O enunciado pede um vídeo "apresentando o protótipo e suas funcionalidades". O pitch de 3 min do prêmio da GS é outro vídeo e não entra aqui.

Bloco	Tempo	Quem fala	Na tela
1. Abertura e problema	0:00 – 0:30	Rafael	Dashboard (ou capa do relatório)
2. Arquitetura	0:30 – 1:20	David	`arquitetura_sistema.png`
3. Demonstração: testes 1 a 4	1:20 – 2:55	Rafael	Wokwi + dashboard lado a lado
4. Previsão com IA	2:55 – 3:35	David	Dashboard (card de espera) → `comparacao_previsao.png`
5. Análise da arquitetura	3:35 – 4:25	Rafael	Serial com o comando `s` → `memoria_esp32.png`
6. Encerramento	4:25 – 4:40	David	Dashboard (rodapé com o aviso)

Antes de gravar (checklist)

- ☐ Firmware **v2** colado e salvo no Wokwi (o cabeçalho do código fala em "Nucleo 1 / Nucleo 0").
- ☐ Apagar `data/eventos.csv` para o dashboard começar limpo, sem os eventos de teste.
- ☐ Backend rodando (`python -m backend.app`) e com o selo **MQTT conectado** verde.
- ☐ Wokwi rodando, com `[MQTT] conectado` no Serial e o dashboard mostrando **ESP32 online**.
- ☐ Tela dividida: Wokwi à esquerda, dashboard à direita. Zoom do navegador em 110–125% para ficar legível no vídeo.
- ☐ Abas abertas com os 3 PNGs: `docs/diagramas/arquitetura_sistema.png`, `docs/relatorio/comparacao_previsao.png` e `docs/diagramas/memoria_esp32.png`.
- ☐ Ensaiar uma vez cronometrando. Se passar de 4:50, cortar trechos marcados com *(opcional)*.
- ☐ Gravar em 1080p (OBS, que vocês já usaram) e subir no YouTube como **não listado**. Colar o link em `docs/video_url.txt` e no README.

> Dica: se o Wokwi ou o broker falharem na hora, usem o plano B, `python simulador/simular_esp32.py` (modo manual: `c` / `a`). Nesse caso, falem que é o simulador.

---

1. Abertura e problema (0:00 – 0:30) · Rafael

Tela: dashboard com a fila vazia.

> "Oi, eu sou o Rafael, RM 569729, e esse é o David, RM 568938. Esse é o ThriveQueue, o nosso projeto de Computer Organization and Architecture da Global Solution: um sistema de fila virtual para o Citizen Assist AI. A ideia é simples: quem chega num posto de atendimento não sabe quantas pessoas estão na frente nem quanto vai esperar. O nosso sistema conta a fila com botões, estima a espera com inteligência artificial e mostra tudo na internet."

2. Arquitetura (0:30 – 1:20) · David

Tela: `arquitetura_sistema.png`. Apontar com o mouse cada bloco enquanto fala.

> "O cérebro é um ESP32 simulado no Wokwi, com processador Xtensa de dois núcleos a 240 MHz. À esquerda estão as **entradas**: dois botões, um de chegada e um de atendimento concluído, ligados em GPIO com pull-up interno. À direita, as **saídas**: um LCD pelo barramento I^2C e dois LEDs, verde para operação válida e vermelho para inválida. Dentro do chip, a **SRAM** de 520 KB guarda o contador e um buffer com os últimos 100 eventos, e a **flash** de 4 MB guarda o programa. E embaixo, a **comunicação**: o ESP32 usa Wi-Fi e manda cada evento por MQTT para um broker na internet. Um backend em Python recebe, calcula a previsão de espera e devolve para o LCD. E o dashboard consulta o backend por HTTP."

3. Demonstração: testes 1 a 4 (1:20 – 2:55) · Rafael

Tela: Wokwi + dashboard lado a lado.

Teste 1 — chegadas (apertar o botão verde 3 vezes, com calma)

> "Teste 1: chegada de cidadãos. Vou apertar o botão de chegada três vezes... O LCD mostra Fila 3, o LED verde pisca a cada clique e o dashboard atualiza sozinho. Cada evento fica registrado com horário, aqui na lista e no Serial."

Teste 2 — atendimentos (botão azul 2 vezes)

> "Teste 2: conclusão de atendimentos. Dois cliques no azul... a fila cai para 1 e os concluídos sobem para 2."

Teste 3 — fila vazia (azul até zerar e mais uma vez)

> "Teste 3, o mais importante: tentar tirar alguém de uma fila vazia. Zero a fila... e aperto de novo. A fila **não fica negativa**: o LED vermelho acende, o LCD avisa 'FILA VAZIA' e o sistema registra uma saída inválida, que aparece em vermelho no dashboard."

Teste 4 — estimativa (verde 3 vezes devagar, apontando para o LCD e o card de espera)

> "Teste 4: atualização da estimativa. A cada chegada, o backend recalcula a espera e manda de volta: 6 minutos, 12, 18... Ela aparece no LCD e no dashboard em menos de um segundo."

(opcional, 10 s) Apertar os dois botões juntos:

> "E se alguém apertar os dois botões juntos, o sistema ignora, porque a operação é ambígua."

4. Previsão com IA (2:55 – 3:35) · David

Tela: card "Tempo estimado de espera" (barras IA × Média) e depois `comparacao_previsao.png`.

> "Para a previsão comparamos dois métodos. O convencional multiplica as pessoas na fila pela duração média de um atendimento. O de IA é uma regressão linear que também considera a duração dos últimos atendimentos e o horário. Treinamos com um histórico **simulado** de quase 2 mil atendimentos e testamos em dias que o modelo nunca viu. A regressão errou em média 2,1 minutos, contra 2,85 da média, ou seja, 26% menos. E a vantagem aumenta quando a fila está maior, que é quando a informação mais importa."

5. Análise da arquitetura (3:35 – 4:25) · Rafael

Tela: digitar `s` no Serial do Wokwi e depois mostrar `memoria_esp32.png`.

> "Com o comando 's' o próprio ESP32 mostra os números para a análise: o programa ocupa 963 KB da flash, sobram mais de 200 KB de heap e cada evento ocupa só 8 bytes. Quem mais gasta memória é a pilha de rede, não a fila. Medindo o tempo do loop, achamos um problema na primeira versão: quando o MQTT reconectava, o loop travava por **4,4 segundos**, e um clique nesse momento seria perdido. A solução foi usar os **dois núcleos**: a rede foi para uma tarefa no núcleo 0 e os botões ficaram sozinhos no núcleo 1, com um mutex protegendo o buffer compartilhado. O pior caso caiu de 4,4 segundos para 0,1."

(opcional, 10 s)

> "Se a internet cair, a fila continua funcionando e os eventos são enviados quando a conexão volta."

6. Encerramento (4:25 – 4:40) · David

Tela: rodapé do dashboard com o aviso.

> "Então o ThriveQueue conta a fila sem erros, prevê a espera com IA e mostra tudo na internet. Lembrando que é um protótipo acadêmico, não é um canal oficial, e não coleta nenhum dado pessoal. Valeu!"

Se precisar cortar tempo

1. Tirar os dois trechos *(opcional)*: economiza ~20 s.
2. No bloco 2, falar só dos blocos maiores (entradas, ESP32, saídas, internet): economiza ~15 s.
3. No teste 4, mostrar só duas chegadas: economiza ~10 s.